

Лабораторна робота

Дисципліна: Веб-технології та дизайн

Тема: Створення базової CRM-системи з адміністративною панеллю у Laravel

Мета: Навчитись створювати адміністративний інтерфейс для керування клієнтами та задачами, реалізувати CRUD-операції, захистити доступ за допомогою авторизації та розмежування ролей.

Завдання

Розробити базову CRM-систему, яка містить:

1. Авторизацію користувачів (Laravel Breeze або Laravel UI).
 2. Ролі: **адміністратор** та **менеджер**.
 3. Сторінку списку клієнтів (тільки для авторизованих).
 4. CRUD-операції для клієнтів (тільки адміністратор).
 5. CRUD-операції для задач (кожен менеджер бачить лише свої задачі).
 6. Панель адміністратора з навігацією.
-

Етапи виконання

Крок 1: Створення проєкту Laravel

```
laravel new crm-panel
```

```
cd crm-panel
```

```
php artisan migrate
```

Крок 2: Встановлення Breeze (для авторизації)

```
composer require laravel/breeze --dev
```

```
php artisan breeze:install
```

```
npm install && npm run dev
```

```
php artisan migrate
```

Крок 3: Створення моделей та міграцій

```
php artisan make:model Client -m
```

```
php artisan make:model Task -m
```

clients міграція:

```
Schema::create('clients', function (Blueprint $table) {
```

```
    $table->id();
```

```
    $table->string('name');
```

```
    $table->string('email')->unique();
```

```
$table->string('phone')->nullable();

$table->timestamps();

});
```

tasks міграція:

```
Schema::create('tasks', function (Blueprint $table) {

    $table->id();

    $table->string('title');

    $table->text('description')->nullable();

    $table->foreignId('client_id')->constrained()->onDelete('cascade');

    $table->foreignId('user_id')->constrained()->onDelete('cascade'); //
```

менеджер

```
$table->enum('status', ['нове', 'в процесі', 'завершено'])->default('нове');

$table->timestamps();

});
```

php artisan migrate

Крок 4: Додавання ролей

У **users** додати колонку role:

```
php artisan make:migration add_role_to_users_table --table=users
```

```
Schema::table('users', function (Blueprint $table) {  
  
    $table->string('role')->default('manager'); // або 'admin'  
  
});
```

```
php artisan migrate
```

Крок 5: Middleware для перевірки ролей

```
php artisan make:middleware RoleMiddleware
```

```
public function handle($request, Closure $next, $role)  
  
{  
  
    if (auth()->check() && auth()->user()->role === $role) {  
  
        return $next($request);  
  
    }  
  
    abort(403);  
  
}
```

Зареєструй middleware в `Kernel.php`, наприклад:

```
'role' => \App\Http\Middleware\RoleMiddleware::class,
```

Крок 6: CRUD для клієнтів і задач

```
php artisan make:controller ClientController --resource
```

```
php artisan make:controller TaskController --resource
```

- Адмін має доступ до всіх клієнтів
- Менеджер — тільки до своїх задач

У `routes/web.php`:

```
Route::middleware(['auth'])->group(function () {
```

```
    Route::middleware('role:admin')->group(function () {
```

```
        Route::resource('clients', ClientController::class);
```

```
    });
```

```
    Route::resource('tasks', TaskController::class)->middleware('role:manager');
```

```
});
```

Крок 7: Шаблон панелі

Можна використати готову адмінку на Bootstrap або Tailwind (на вибір).

Приклад головної навігації в

<resources/views/layouts/app.blade.php>:

```
<nav>
```

```
    <a href="{{ route('clients.index') }}">Клієнти</a>
```

```
    <a href="{{ route('tasks.index') }}">Задачі</a>
```

```
</nav>
```

Звіт до лабораторної роботи

У звіті вкажіть:

- Ваше прізвище, групу, дату.
- Скріншоти:
 - реєстрації та входу;
 - списку клієнтів;
 - списку задач;

- створення задачі;
 - панелі адміністратора.
 - Код міграцій, контролерів та views.
 - Висновок: які труднощі виникли, що дізналися нового.
-

Результат

Після завершення роботи студент має отримати базову CRM-систему з адмін-панеллю, яка демонструє:

- Ролі користувачів;
- Реалізацію авторизації;
- Захист маршрутів;
- CRUD для двох сутностей.

Варіанти завдань до лабораторної роботи «CRM на Laravel»

Варіант 1. CRM для технічної підтримки

- Сутності: **Користувач, Звернення, Коментарі**
 - Кожен менеджер бачить тільки звернення, які йому призначені
 - Адмін бачить усі звернення та може змінювати статус: "нове", "в роботі", "закрито"
-

Варіант 2. CRM для фріланс-біржі

- Сутності: **Клієнт, Проєкт, Менеджер**
 - Кожен проєкт має бюджет і дедлайн
 - Менеджер може бачити тільки свої проєкти
-

Варіант 3. CRM для кур'єрської служби

- Сутності: **Замовлення, Клієнт, Кур'єр**
 - Адмін розподіляє замовлення між кур'єрами
 - Кур'єр бачить лише свої доставлення
-

Варіант 4. CRM для клініки

- Сутності: **Пацієнт, Запис на прийом, Лікар**
 - Пацієнти додаються адміністратором
 - Лікар бачить лише свої записи
-

Варіант 5. CRM для агентства нерухомості

- Сутності: **Клієнт, Об'єкт, Менеджер**
 - Кожен об'єкт має статус: "активний", "в процесі", "продано"
 - Менеджер може додавати нотатки по об'єктах
-

Варіант 6. CRM для школи

- Сутності: **Учень, Завдання, Викладач**
 - Учні не мають входу в систему — керування здійснює викладач
 - Завдання мають термін виконання і оцінку
-

Варіант 7. CRM для IT-компанії

- Сутності: **Клієнт, Тікет, Інженер**
 - Менеджер створює тікети, інженер обробляє
 - Статуси: "відкрито", "призначено", "вирішено"
-

Варіант 8. CRM для фотостудії

- Сутності: **Клієнт, Фотосесія, Фотограф**
 - Фотосесія має дату, тривалість, тип (весільна, сімейна і т.д.)
 - Адмін призначає фотографа
-

Варіант 9. CRM для логістичної компанії

- Сутності: **Водій, Рейс, Маршрут**
 - Адмін додає водіїв і маршрути
 - Водій бачить тільки свої рейси
-

Варіант 10. CRM для фітнес-клубу

- Сутності: **Клієнт, Тренування, Тренер**
- Тренер бачить список своїх клієнтів і тренування
- Адмін призначає клієнтів тренерам